

A Very Fast String Matching Algorithm Based on Condensed Alphabets

Simone Faro^(✉)

Università di Catania, Viale A.Doria n.6, 95125 Catania, Italy
faro@dmf.unict.it

Abstract. String matching is the problem of finding all the substrings of a text which correspond to a given pattern. It's one of the most investigated problem in computer science, mainly due to its various applications in many fields. In recent years most solutions to the problem focused on efficiency and flexibility of the searching procedure and effective techniques appeared to speed-up previous solutions. In this paper we present a simple and very efficient algorithm for string matching. It can be seen as an extension of the Skip-Search algorithm to condensed alphabets with the aim of reducing the number of verifications during the searching phase. From our experimental results it turns out that the new variant obtains in most cases the best running time when compared against the most effective algorithms in literature. This makes the new algorithm one of the most flexible solutions in practical cases.

Keywords: Exact text analysis · String matching · Experimental algorithms · Text processing

1 Introduction

The *exact string matching problem* is one of the most studied problem in computer science. It consists in finding all the (possibly overlapping) occurrences of an input pattern x in a text y , over the same alphabet Σ of size σ . A huge number of solutions have been devised since the 1980s [6, 17] and, in spite of such wide literature, much work has been produced in the last few years, indicating that the need for efficient solutions to this problem is high.

Solutions to such problem can be divided in two classes, *counting* solutions simply counts the number of occurrences of the pattern in the text, while *reporting* solutions are also able to report the exact positions in which the pattern occurs. Solutions in the first class are faster in general. In this paper we are interested in the second class of algorithms.

From a theoretical point of view the exact string matching problem has been extensively studied. If we indicate with m and n the lengths of the pattern and of the text, respectively, the problem can be solved in $\mathcal{O}(n)$ worst case time

This work has been supported by G.N.C.S., Istituto Nazionale di Alta Matematica “Francesco Severi”.

complexity [20]. However, in many practical cases it is possible to avoid reading all the characters of the text achieving sub-linear performances on the average. The optimal average $\mathcal{O}(\frac{n \log_{\sigma} m}{m})$ time complexity [23] was reached for the first time by the Backward-DAWG-Matching algorithm [8] (BDM). Interested readers can refer to [6, 14, 17] for a survey of the most efficient solutions to the problem.

In recent years most solutions to the problem focused on efficiency and flexibility and effective techniques appeared to speed-up previous formerly efficient solutions. Among such techniques *bit-parallelism*, *string-packing*, *q-grams*, *filtering* and *hashing* deserve a special mention since they inspired a lot of work. *Filtering* and *hashing* are two techniques particularly relevant in this paper.

Specifically, instead of checking at each position of the text if the pattern occurs, it seems to be more efficient to *filter* positions of the text by checking only if the corresponding content *looks like* the input pattern. When a resemblance is detected a more detailed check is performed.

The first algorithm to take advantage of such technique was the well known Karp-Rabin algorithm [19] in 1987. It uses *hashing* function for computing a fingerprint value of the pattern. Subsequently a fingerprint value for each text substring of length m is computed. Then a naive check at a given position of the text is performed only if the fingerprint value of the corresponding substring is equal to the fingerprint value of the pattern. The overall worst case time complexity of the algorithm is $\mathcal{O}(nm)$ but a linear behavior can be observed on average. The first solution based on filtering and hashing, showing a sub-linear average behavior, was presented by Lecroq in 2007 [21]. The algorithm, named Hash q , is simply a generalization of the Boyer-Moore-Horspool algorithm to condensed alphabets. In this case groups of q characters (or q -grams) are hashed in a single fingerprint value, generating an extended condensed alphabet. Such condensed alphabet correspond to the set of all fingerprint values generated by all possible combinations of q characters drawn from the original alphabet.

The idea of extending efficient solutions by condensed alphabets has been later extensively adopted in string matching [17]. However, although several algorithms have been proposed in the last decade, the Hash q algorithm is still one of the most effective solutions in practical cases [14].

In this paper we present a simple, yet very efficient, algorithm for the exact string matching problem based on a well known filtering solution, the Skip-Search algorithm [7], extended with condensed alphabets. We will observe how the use of a condensed alphabet allows to drastically reduce the number of verifications of such filtering approach. The worst case time complexity of the algorithm is $\mathcal{O}(nm)$. However, despite its quadratic worst case behavior, we will show in our experimental evaluation that such extension leads to one of the most efficient and flexible algorithms for string matching. Specifically it turns out that the new solution obtains the best results, in terms of running times, in most cases and especially for small alphabets and long patterns.

The paper is organized as follows. In Sect. 2 we describe in detail the Skip-Search algorithm and its variants. In Sect. 3 we introduce and analyze the new algorithm based on condensed alphabets. In Sect. 4 we compare the new

presented algorithm against the most effective solutions known in literature. We draw our conclusions in Sect. 5.

2 The Skip Search and the Alpha Skip Search Algorithms

The Skip Search algorithm is an elegant and efficient solution to the exact pattern matching problem, firstly presented in [7] and subsequently adapted to many other problems and variants of exact pattern matching.

Let x and y be a pattern and a text of length m and n , respectively, over a common alphabet Σ of size σ . For each character c of the alphabet, the Skip Search algorithm collects in a bucket $B[c]$ all the positions of that character in the pattern x , so that for each $c \in \Sigma$ we have:

$$B[c] = \{i : 0 \leq i \leq m - 1 \text{ and } x[i] = c\}.$$

Plainly, the space and time complexity needed for the construction of the array B of buckets is $\mathcal{O}(m + \sigma)$.

Thus if a character occurs k times in the pattern, there are k corresponding positions in the bucket of the character. Notice that when the pattern is shorter than the alphabet size, some buckets are empty. This observation turns out to be particularly suitable for our purpose.

The search phase of the Skip Search algorithm examines all the characters $y[j]$ in the text at positions $j = km - 1$, for $k = 1, 2, \dots, \lfloor n/m \rfloor$. For each such character $y[j]$, the bucket $B[y[j]]$ allows one to compute the possible positions h of the text in the neighborhood of j at which the pattern could occur.

By performing a character-by-character comparison between x and the substring $y[h..h + m - 1]$ until either a mismatch is found, or all the characters in the pattern x have been considered, it can be tested whether x actually occurs at position h of the text.

The Skip Search algorithm has a quadratic worst-case time complexity, however, as shown in [7], the expected number of text character inspections is $\mathcal{O}(n)$. In addition it is interesting to observe that the Skip Search algorithms performs better in the case of large alphabets since most of the buckets in the array B are empty.

Among the variants of the Skip Search algorithm, the most relevant one for our purposes is the Alpha Skip Search algorithm [7], which collects buckets for substrings of the pattern rather than for its single characters.

During the preprocessing phase of the Alpha Skip Search algorithm, all the factors of length $\ell = \lfloor \log_{\sigma} m \rfloor$ occurring in the pattern x are arranged in a trie T_x , for fast retrieval. In addition, for each leaf ν of T_x a bucket is maintained which stores the positions in x of the factor corresponding to ν . Provided that the alphabet size is considered as a constant, the worst-case running time of the preprocessing phase is linear.

The searching phase consists in looking into the buckets of the text factors $y[j..j + \ell - 1]$, for all $j = k(m - \ell + 1) - 1$ such that $1 \leq k \leq \lfloor (n - \ell)/m \rfloor$, and

then test, as in the previous case, whether there is an occurrence of the pattern at the indicated positions of the text.

The worst-case time complexity of the searching phase is quadratic, though the expected number of text character comparisons is $\mathcal{O}(n \log_\sigma m / (m - \log_\sigma m))$.

3 A New Fast Variant of the Skip-Search Algorithm

In this section we present an efficient extension of the Skip-Search algorithm using condensed alphabets. The resulting algorithm has a quadratic worst case time complexity while on average it shows a sublinear behavior.

Let x be a pattern of length m and let y be a text of length n . Moreover suppose both strings are drawn from a common alphabet Σ of size σ and suppose q is a constant value, with $1 \leq q \leq m$. The algorithm can be divided in a preprocessing and a searching phase.

The preprocessing phase of the algorithm indexes all subsequences of the pattern (of length q) in order to be able to locate them during the searching phase. For efficiency reasons, each substring of length q is converted into a numeric value, called *fingerprint*, which is used to index the substring. A fingerprint value ranges in the interval $\{0 \dots 2^\alpha - 1\}$, for a given bound α . In our setting the value α is set to 16, so that a fingerprint can fit into a single 16-bit register.

The procedure FNG for computing the fingerprints is shown in Fig. 1 (on the left). Given a sequence x of length m , an index i such that $0 \leq i < m - q$, and two integers k and q such that $kq \leq \alpha$, the procedure FNG computes the fingerprint v of the substring $x[i \dots i + q - 1]$. Specifically the fingerprint v is computed as

$$v = \sum_{j=0}^{q-1} (x[i+j] \ll kj).$$

Plainly, the time complexity of the procedure FNG is $\mathcal{O}(q)$. Observe that the fingerprint value is not unique for each substring of length q , i.e. two different strings can be associated with the same fingerprint value. The preprocessing phase of the algorithm, which is reported in Fig. 1 (on the left), consists in compiling the fingerprints of all possible substrings of length q contained in the pattern x . Thus a fingerprint value v , with $0 \leq v < 2^\alpha$, is computed for each subsequence $x[i \dots i + q - 1]$, for $0 \leq i < m - q$. To this purpose a table F of size 2^α is maintained for storing, for any possible fingerprint value v , the set of positions i such that $\text{FNG}(x, i, q, k) = v$. More precisely, for $0 \leq v < 2^\alpha$, we have

$$F[v] = \left\{ i \mid 0 \leq i < m - q \text{ and } \text{FNG}(x, i, q, k) = v \right\}.$$

The preprocessing phase of the algorithm requires some additional space to store the $(m - q)$ possible alignments in the 2^α locations of the table F . Thus, the space requirement of the algorithm is $\mathcal{O}(m - q + 2^\alpha)$ that approximates to $\mathcal{O}(m)$, since α is constant. The first loop of the preprocessing phase just initializes the

<pre> FNG(x, i, q, k) 2. $v \leftarrow 0$ 3. for $j \leftarrow q - 1$ downto 0 do 4. $v \leftarrow (v \ll k) + x[i + j]$ 5. return v PREPROCESSING(x, q, m, k) 1. for $v \leftarrow 0$ to $2^\alpha - 1$ do 2. $F[v] \leftarrow \emptyset$ 3. for $i \leftarrow 0$ to $m - q$ do 4. $v \leftarrow \text{FNG}(x, i, q, k)$ 5. $F[v] \leftarrow F[v] \cup \{(i + q - 1)\}$ 6. return F </pre>	<pre> SKIPq(x, r, y, n, q, k) 1. $F \leftarrow \text{Preprocessing}(x, q, m, k)$ 2. for $j \leftarrow m - 1$ to n step $m - q + 1$ do 3. $v \leftarrow \text{FNG}(y, j, q, k)$ 4. for each $i \in F[v]$ do 5. if $x = y[j - i .. j - i + m - 1]$ 6. then output $(j - i)$ </pre>
--	---

Fig. 1. The pseudo-code of the Skip q algorithm for the exact string matching problem.

table F , while the second loop is run $(m - q)$ times, which makes the overall time complexity of such phase $\mathcal{O}(m + 2^\alpha)$ that, again, approximates to $\mathcal{O}(m)$.

Along the same line of the Skip Search algorithm, the basic idea of the searching phase is to compute a fingerprint value every $(m - q + 1)$ positions of the text y and to check whether the pattern appears in y , involving the block $y[j .. j + q - 1]$. If the fingerprint value indicates that some of the alignments are possible, then the candidate positions are checked naively for matching.

The pseudo-code provided in Fig. 1 (on the right) reports the skeleton of the algorithm. The main loop investigates the blocks of the text y in steps of $(m - q + 1)$ blocks. If the fingerprint v computed on $y[j .. j + q - 1]$ points to a nonempty bucket of the table F , then the positions listed in $F[v]$ are verified accordingly.

In particular $F[v]$ contains a linked list of the values i marking the pattern x and the beginning position of the pattern in the text. While looking for occurrences on $y[j .. j + q - 1]$, if $F[v]$ contains the value i , this indicates the pattern x may potentially begin at position $(j - i)$ of the text. In that case, a matching test is to be performed between x and $y[j - i .. j - i + m - 1]$ via a character-by-character inspection.

The total number of filtering operations is exactly $n/(m - q)$. At each attempt, the maximum number of verification requests is $(m - q)$, since the filter provides information about that number of appropriate alignments of the pattern. On the other hand, if the computed fingerprint points to an empty location in F , then there is obviously no need for verification. The verification cost for a pattern x of length m is assumed to be $\mathcal{O}(m)$, with the brute-force checking approach. Hence, in the worst case the time complexity of the verification is $\mathcal{O}(m(m - q))$, which happens when all alignments in x must be verified at any possible beginning

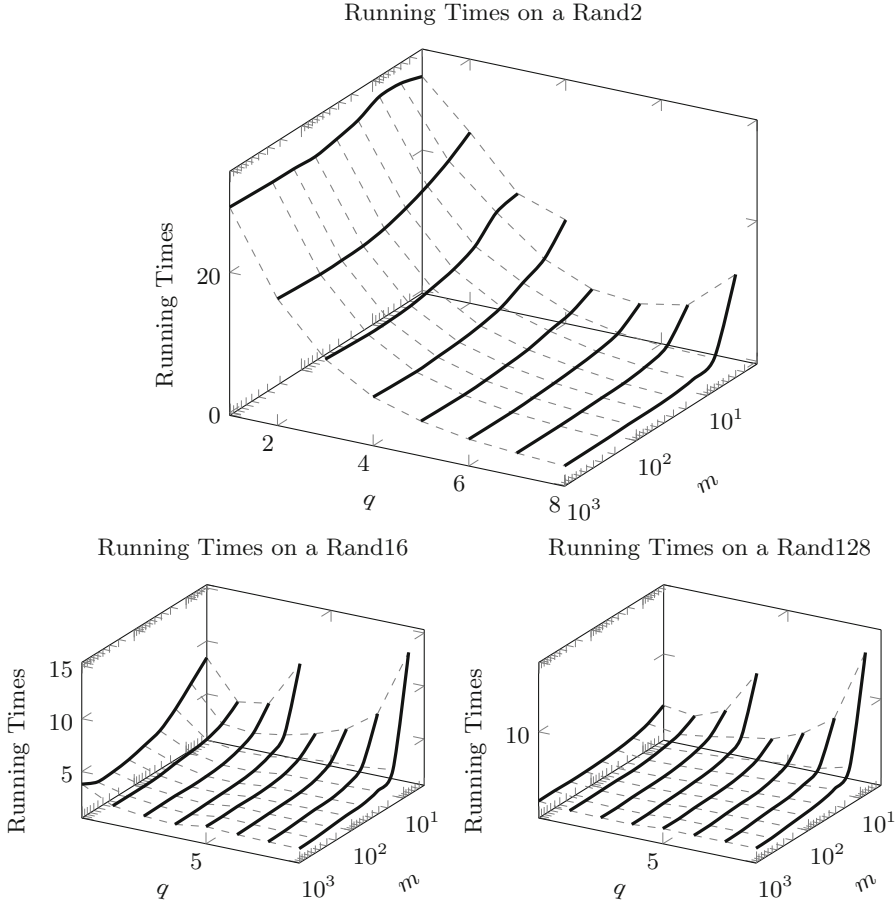


Fig. 2. Running times of the Skip-Search extended with condensed alphabets using groups of q characters. We report running times of the algorithms for different values of q . Experimental test have been conducted on random text over alphabets of size 2, 16 and 128, respectively, with a uniform distribution of characters.

position. Hence, the best case complexity is $\mathcal{O}(n/(m - q))$, while the worst case complexity is $\mathcal{O}(nm)$.

Figure 2 shows experimental evaluations to compare the performances of the algorithm under various conditions and for different values of the parameter q . Experimental evaluations have been conducted on random text of 4Mb over alphabets of size 2, 16 and 128, respectively, with a uniform distribution of characters (a detailed description of the experimental settings can be found in Sect. 4).

It turns out from experimental evaluations shown in Fig. 2 that the performances of the algorithm strongly depend on the values of m , q and σ . When the size of the alphabet is small then larger values of the parameter q are more

effective. Such difference is less sensible when the size of the alphabet gets larger. However it turns out that the smaller is the length of the pattern the lower is the performance of the algorithm. This behavior is more evident for larger values of the parameter q . Thus, the choice of the parameter q should be directed to larger values when the size of alphabet decreases or when the length of the pattern increases. Conversely the values of q should get smaller.

4 Experimental Results

In this section we evaluate the performance of the new presented algorithms against the most efficient solution known in literature for the online exact string matching problem. Specifically we compare the following 15 algorithms implemented in 79 variants, depending on the values of their parameters:

- AOSO q : the Average-Optimal variant [18] of the Shift-Or algorithm [2] using q -grams, with $1 \leq q \leq 6$;
- BN DMq : the Backward-Nondeterministic-DAWG-Matching algorithm [22] implemented using q -grams with $1 \leq q \leq 8$;
- BS DMq : the Backward-SNR-DAWG-Matching algorithm [15] using condensed alphabets with groups of q characters, with $1 \leq q \leq 8$;
- BX Sq : the Backward-Nondeterministic-DAWG-Matching algorithm [22] with Extended Shift [10] implemented using q -grams and $1 \leq q \leq 8$;
- EBOM: the extended version [13] of the BOM algorithm [1];
- FSBN $DMqs$: the Forward Simplified version [13] of the BN DM algorithm [22] implemented using q -grams and $1 \leq q \leq 8$;
- KB NM : the Factorized variant [5] BN DM algorithm [22];
- SB NMq : the Simplified version of the Backward-Nondeterministic-DAWG-Matching algorithm [1] implemented using q -grams and $1 \leq q \leq 8$;
- FS- w : the Multiple Windows version [16] of the Fast Search algorithm [3] implemented using w sliding windows, with $2 \leq w \leq 6$;
- HASH q : the Hashing algorithm [21] using q -grams, with $3 \leq q \leq 5$;
- IOM: the Improved Occurrence Matcher [4];
- WOM: the Worst Occurrence Matcher [4];
- JOM: the Jumping Occurrence Matcher [4];
- ASKIP the Alpha variant of the SKip-Search algorithm [7];
- SKIP q : the new Skip Search variants using q -grams, with $1 \leq q \leq 8$ (observe that when $q = 1$ we have the original Skip-Search algorithm [7]);

For the sake of completeness we evaluate also the following two string matching algorithms for *counting* occurrences.

- EPSM: the Exact Packed String Matching algorithm [12];
- TSO q : the Two-Way variant of [9] the Shift-Or algorithm [2] implemented with a loop unrolling of q characters, with $q = 5$;

All algorithms have been implemented in the C programming language and have been tested using the SMART tool¹. The experiments were executed locally on an MacBook Pro with 4 Cores, a 2 GHz Intel Core i7 processor, 16 GB RAM 1600 MHz DDR3, 256 KB of L2 Cache and 6 MB of Cache L3. Algorithms have been compared in terms of running times, including any preprocessing time.

We report experimental evaluations on a three random sequences (see Tables 1 and 2) and on three real data (see Tables 3, 4 and 5). Specifically random sequences are over alphabets of 2 and 16 characters, with a uniform distribution. For the case of real data evaluations we used a genome sequence, a protein sequence and an english text. All sequences have a length of 5 MB, are provided by the SMART research tool and are available online for download.

During the experimental evaluations patterns of length m were randomly extracted from the sequences, with m ranging over the set of values $\{2^i \mid 2 \leq i \leq 10\}$. For each case, the mean over the running times, expressed in hundredths of seconds, of 500 runs has been reported.

In the following tables we report the running times of our evaluations. Each table is divided in four blocks. The first and the second block present the most effective algorithms known in literature based on automata and comparison of characters, respectively. Best results among this two sets of algorithms have been bold-faced in order to easily locate the best solutions among previous known algorithms. The third block contains the running times of the new algorithm, including the speed up (in percentage) obtained against the best running time in the first two blocks. Positive values indicate a braking of the running time while

Table 1. Experimental results on a random sequence over an alphabet of 2 characters.

m	4	8	16	32	64	128	256	512	1024
AOSO q	41.70 ⁽²⁾	35.62 ⁽⁴⁾	14.37 ⁽⁴⁾	4.54 ⁽⁴⁾	4.64 ⁽⁴⁾	4.59 ⁽⁴⁾	4.58 ⁽⁴⁾	4.62 ⁽⁴⁾	4.57 ⁽⁴⁾
BNDM q	16.55 ⁽⁴⁾	8.67 ⁽⁶⁾	5.10 ⁽⁶⁾	4.06 ⁽⁶⁾	4.92 ⁽⁴⁾	4.92 ⁽⁴⁾	4.92 ⁽⁴⁾	4.89 ⁽⁴⁾	4.90 ⁽⁴⁾
BSDM q	15.35⁽⁴⁾	7.52⁽⁶⁾	4.30⁽⁸⁾	3.30⁽⁸⁾	2.99⁽⁸⁾	2.79⁽⁸⁾	2.78⁽⁸⁾	2.74 ⁽⁸⁾	2.73 ⁽⁸⁾
BXS q	22.33 ⁽⁴⁾	10.97 ⁽⁶⁾	5.47 ⁽⁸⁾	3.74 ⁽⁸⁾	3.75 ⁽⁸⁾	3.75 ⁽⁸⁾	3.76 ⁽⁸⁾	3.74 ⁽⁸⁾	3.75 ⁽⁸⁾
EBOM	25.12	20.32	13.65	8.72	6.27	4.62	3.77	3.28	2.98
FSBNDM qs	16.66 ^(4,1)	7.80 ^(6,1)	5.08 ^(6,1)	4.05 ^(6,1)	4.00 ^(6,1)	4.07 ^(6,1)	4.05 ^(6,1)	4.05 ^(6,1)	4.05 ^(6,1)
KBNDM	27.67	19.08	11.31	7.07	5.82	5.71	5.75	5.78	5.79
SBNDM q	15.36 ⁽⁴⁾	8.33 ⁽⁶⁾	5.05 ⁽⁶⁾	4.01 ⁽⁶⁾	4.05 ⁽⁶⁾	4.08 ⁽⁶⁾	4.07 ⁽⁶⁾	4.08 ⁽⁶⁾	4.09 ⁽⁶⁾
FS- w	26.30 ⁽²⁾	20.20 ⁽²⁾	15.16 ⁽²⁾	11.63 ⁽²⁾	9.42 ⁽²⁾	8.00 ⁽²⁾	6.90 ⁽²⁾	6.11 ⁽²⁾	5.52 ⁽²⁾
FJS	28.78	33.80	36.59	34.94	36.25	36.02	36.51	36.36	36.55
HASH q	25.44 ⁽³⁾	11.94 ⁽⁵⁾	6.07 ⁽⁵⁾	3.99 ⁽⁸⁾	3.20 ⁽⁸⁾	2.97 ⁽⁸⁾	2.95 ⁽⁸⁾	2.71⁽⁸⁾	2.62⁽⁸⁾
ASKIP	34.24	22.43	13.14	6.50	4.62	3.62	3.27	3.16	3.52
IOM	23.90	24.64	26.64	26.58	26.76	26.60	26.69	26.51	26.67
WOM	30.32	26.31	23.00	20.37	17.94	16.47	14.94	13.78	12.76
SKIP q	14.37⁽⁴⁾	7.24⁽⁶⁾	4.30⁽⁸⁾	3.20⁽⁸⁾	2.77⁽⁸⁾	2.65⁽⁸⁾	2.65⁽⁸⁾	2.55⁽⁸⁾	2.51⁽⁸⁾
speed-up	-6.4 %	-3.7 %	0.0 %	-3.0 %	-7.3 %	-5.0 %	-4.7 %	-5.9 %	-4.2 %
EPSM	6.51	8.46	3.20	2.45	2.25	2.22	2.30	2.31	2.30
TSO q	12.67 ⁽⁵⁾	10.01 ⁽⁵⁾	6.67 ⁽⁵⁾	4.39 ⁽⁵⁾	3.17 ⁽⁵⁾	-	-	-	-

¹ The SMART tool is available online at <http://www.dmi.unict.it/~faro/smart/>.

Table 2. Experimental results on a random sequence over an alphabet of 16 characters.

m	4	8	16	32	64	128	256	512	1024
AOSO q	10.70 ⁽²⁾	4.29 ⁽⁴⁾	3.75 ⁽⁴⁾	3.75 ⁽⁴⁾	3.07 ⁽⁶⁾	3.11 ⁽⁶⁾	3.13 ⁽⁶⁾	3.09 ⁽⁶⁾	3.14 ⁽⁶⁾
BNDM q	11.99 ⁽⁴⁾	4.20 ⁽⁴⁾	2.96 ⁽⁴⁾	2.38 ⁽⁴⁾	2.39 ⁽⁴⁾	2.41 ⁽⁴⁾	2.38 ⁽⁴⁾	2.35 ⁽⁴⁾	2.39 ⁽⁴⁾
BSDM q	4.82 ⁽²⁾	3.79 ⁽⁴⁾	2.67 ⁽⁴⁾	2.27 ⁽⁴⁾	2.10⁽⁴⁾	2.02⁽⁴⁾	1.98⁽⁴⁾	1.96⁽⁴⁾	1.99⁽⁴⁾
BXS q	6.86 ⁽²⁾	4.29 ⁽²⁾	3.08 ⁽²⁾	2.46 ⁽²⁾	2.47 ⁽⁴⁾	2.50 ⁽⁴⁾	2.51 ⁽⁴⁾	2.52 ⁽⁴⁾	2.48 ⁽⁴⁾
EBOM	3.75	2.92	2.51	2.25	2.16	2.19	2.16	2.16	2.30
FSBNDM qs	4.28 ^(2,0)	3.21 ^(2,0)	2.55 ^(3,1)	2.18 ^(3,1)	2.20 ^(3,1)	2.18 ^(3,1)	2.19 ^(3,1)	2.19 ^(3,1)	2.21 ^(3,1)
KBNDM	7.38	4.94	3.76	3.17	2.95	3.04	2.90	3.00	2.98
SBNDM q	5.26 ⁽²⁾	3.62 ⁽²⁾	2.73 ⁽²⁾	2.31 ⁽²⁾	2.36 ⁽⁴⁾	2.37 ⁽⁴⁾	2.37 ⁽⁴⁾	2.39 ⁽⁴⁾	2.42 ⁽⁴⁾
FS- w	5.15 ⁽⁶⁾	3.69 ⁽⁶⁾	3.05 ⁽⁶⁾	2.72 ⁽⁶⁾	2.72 ⁽⁶⁾	2.68 ⁽⁶⁾	2.64 ⁽⁶⁾	2.64 ⁽⁶⁾	2.66 ⁽⁶⁾
FJS	9.38	7.08	5.27	4.94	4.65	4.54	4.18	4.65	4.49
HASH q	19.81 ⁽³⁾	8.35 ⁽³⁾	5.07 ⁽³⁾	3.61 ⁽⁵⁾	3.10 ⁽⁵⁾	2.97 ⁽⁵⁾	2.89 ⁽⁵⁾	2.73 ⁽⁵⁾	2.62 ⁽⁵⁾
ASKIP	9.04	7.48	4.98	3.35	2.84	2.82	3.07	3.76	6.23
IOM	8.96	6.40	5.11	4.49	4.35	4.28	4.28	4.31	4.35
WOM	9.39	6.66	5.16	4.43	4.17	3.97	3.85	3.80	3.72
SKIP q	4.85 ⁽²⁾	3.70 ⁽³⁾	3.13 ⁽³⁾	2.24⁽⁴⁾	2.06⁽⁴⁾	1.99⁽⁴⁾	1.93⁽⁴⁾	1.86⁽⁴⁾	1.84⁽⁴⁾
<i>speed-up</i>	+29 %	+26 %	+24 %	-0.4 %	-1.9 %	-1.5 %	-2.5 %	-5.1 %	-7.5 %
EPSM	6.62	25.55	2.72	2.10	1.94	1.89	1.81	1.74	1.78
TSO q	5.45 ⁽⁵⁾	3.88 ⁽⁵⁾	3.11 ⁽⁵⁾	2.51 ⁽⁵⁾	2.20 ⁽⁵⁾	-	-	-	-

Table 3. Experimental results on a genome sequence.

m	4	8	16	32	64	128	256	512	1024
AOSO q	16.98 ⁽²⁾	9.63 ⁽²⁾	3.93 ⁽⁴⁾	3.39 ⁽⁴⁾	2.98 ⁽⁶⁾	2.97 ⁽⁶⁾	2.99 ⁽⁶⁾	3.00 ⁽⁶⁾	3.03 ⁽⁶⁾
BNDM q	11.13 ⁽⁴⁾	4.10 ⁽⁴⁾	2.99 ⁽⁴⁾	2.47 ⁽⁴⁾	2.38 ⁽⁴⁾	2.39 ⁽⁴⁾	2.41 ⁽⁴⁾	2.47 ⁽⁴⁾	2.45 ⁽⁴⁾
BSDM q	8.37 ⁽⁴⁾	3.71⁽⁴⁾	2.78⁽⁴⁾	2.46⁽⁴⁾	2.25⁽⁸⁾	2.15⁽⁸⁾	2.11⁽⁸⁾	2.16⁽⁶⁾	2.11⁽⁶⁾
BXS q	11.86 ⁽²⁾	4.78 ⁽⁴⁾	3.25 ⁽⁴⁾	2.53 ⁽⁶⁾	2.50 ⁽⁶⁾	2.52 ⁽⁴⁾	2.49 ⁽⁴⁾	2.55 ⁽⁴⁾	2.54 ⁽⁴⁾
EBOM	7.72	7.15	5.66	4.10	3.17	2.67	2.40	2.32	2.41
FSBNDM qs	6.46^(3,1)	3.87^(4,1)	2.94^(4,1)	2.38^(4,1)	2.35^(6,2)	2.31^(6,1)	2.33^(6,1)	2.38^(3,1)	2.37^(6,1)
KBNDM	10.88	8.21	6.15	4.17	3.27	3.09	3.10	3.13	3.14
SBNDM q	8.75 ⁽²⁾	3.95 ⁽⁴⁾	2.97 ⁽⁴⁾	2.47 ⁽⁴⁾	2.39 ⁽⁴⁾	2.39 ⁽⁴⁾	2.36 ⁽⁴⁾	2.38 ⁽⁴⁾	2.38 ⁽⁴⁾
FS- w	12.33 ⁽²⁾	9.39 ⁽²⁾	7.76 ⁽²⁾	6.89 ⁽²⁾	6.16 ⁽²⁾	5.63 ⁽²⁾	5.06 ⁽²⁾	4.73 ⁽²⁾	4.42 ⁽²⁾
FJS	18.60	16.69	16.96	15.96	16.09	16.80	16.71	16.61	16.59
HASH q	18.09 ⁽³⁾	7.68 ⁽³⁾	4.67 ⁽⁵⁾	3.31 ⁽⁵⁾	2.78 ⁽⁵⁾	2.60 ⁽⁵⁾	2.63 ⁽⁵⁾	2.51 ⁽⁵⁾	2.40 ⁽⁵⁾
ASKIP	17.19	7.45	4.32	3.17	2.66	2.60	2.79	3.24	5.18
IOM	14.41	11.88	11.08	11.17	11.17	11.13	11.03	11.03	10.98
WOM	16.69	12.48	9.88	8.61	7.75	7.16	6.72	6.29	6.11
SKIP q	6.02⁽³⁾	3.71⁽⁴⁾	2.76⁽⁴⁾	2.34⁽⁴⁾	2.15⁽⁴⁾	2.08⁽⁶⁾	2.01⁽⁸⁾	1.91⁽⁸⁾	1.88⁽⁸⁾
<i>speed-up</i>	-6.8 %	0.0 %	-0.7 %	-4.9 %	-4.4 %	-3.3 %	-4.7 %	-12 %	-11 %
EPSM	5.87	3.72	2.50	1.93	1.75	1.72	1.66	1.62	1.65
TSO q	5.54 ⁽⁵⁾	3.85 ⁽⁵⁾	3.08 ⁽⁵⁾	2.42 ⁽⁵⁾	2.05 ⁽⁵⁾	-	-	-	-

a negative percentage represent and improvements of the performance. Running times with an improvement of the performance have been bold-faced.

The last block reports the running times obtained by the best two algorithms for *counting* occurrences (we do not compare them against the other algorithms).

Among the previous solutions it turns out that the BSDM q algorithm is fastest in the case of small alphabets (2 and 16 characters), however it is second to the Hash q algorithm for $\sigma = 2$ and very long patterns, and to the EBOM algorithm for $\sigma = 16$ and short patterns. Regarding the performance of the new

Table 4. Experimental results on a protein sequence.

m	4	8	16	32	64	128	256	512	1024
AOSO q	10.80 ⁽²⁾	4.27 ⁽⁴⁾	3.84 ⁽⁴⁾	3.81 ⁽⁴⁾	3.18 ⁽⁴⁾	3.17 ⁽⁴⁾	3.16 ⁽⁴⁾	3.16 ⁽⁴⁾	3.16 ⁽⁴⁾
BNDM q	12.20 ⁽⁴⁾	4.29 ⁽⁴⁾	3.06 ⁽⁴⁾	2.46 ⁽⁴⁾	2.45 ⁽⁴⁾	2.43 ⁽⁴⁾	2.42 ⁽⁴⁾	2.40 ⁽⁴⁾	2.40 ⁽⁴⁾
BSDM q	4.68 ⁽²⁾	3.71 ⁽²⁾	2.75 ⁽⁴⁾	2.35 ⁽⁴⁾	<u>2.06⁽⁴⁾</u>	<u>1.98⁽⁴⁾</u>	<u>1.97⁽⁴⁾</u>	<u>1.97⁽⁴⁾</u>	<u>1.94⁽⁴⁾</u>
BXS q	6.91 ⁽²⁾	4.29 ⁽²⁾	3.12 ⁽²⁾	2.52 ⁽²⁾	2.48 ⁽²⁾	2.52 ⁽²⁾	2.50 ⁽²⁾	2.51 ⁽²⁾	2.52 ⁽²⁾
EBOM	3.87	2.94	2.57	2.29	2.11	2.18	2.20	2.24	2.42
FSBNDM qs	4.32 ^(2,0)	3.28 ^(2,0)	2.59 ^(3,1)	2.26 ^(3,1)	2.22 ^(3,1)	2.25 ^(3,1)	2.25 ^(3,1)	2.20 ^(3,1)	2.26 ^(3,1)
KBNDM	7.46	4.97	3.81	3.24	3.04	3.01	2.95	2.96	2.95
SBNDM q	5.25 ⁽²⁾	3.67 ⁽²⁾	2.79 ⁽²⁾	2.34 ⁽²⁾	2.45 ⁽⁴⁾	2.41 ⁽⁴⁾	2.42 ⁽⁴⁾	2.41 ⁽⁴⁾	2.40 ⁽⁴⁾
FS- w	6.18 ⁽²⁾	4.33 ⁽²⁾	3.55 ⁽²⁾	3.20 ⁽²⁾	3.05 ⁽²⁾	2.94 ⁽²⁾	2.90 ⁽²⁾	2.87 ⁽²⁾	2.86 ⁽²⁾
FJS	9.68	18.54	4.18	3.02	2.92	2.89	2.82	3.16	4.11
HASH q	19.92 ⁽³⁾	8.36 ⁽³⁾	5.05 ⁽³⁾	3.75 ⁽⁵⁾	3.19 ⁽⁵⁾	2.99 ⁽⁵⁾	2.92 ⁽⁵⁾	2.76 ⁽⁵⁾	2.66 ⁽⁵⁾
ASKIP	8.63	7.06	5.07	3.52	2.87	2.87	3.13	3.92	6.56
IOM	8.87	6.36	5.02	4.41	4.04	3.92	3.86	3.86	3.79
WOM	9.31	6.61	5.13	4.32	4.03	3.72	3.56	3.43	3.33
SKIP q	4.56 ⁽²⁾	3.41 ⁽³⁾	2.64 ⁽³⁾	<u>2.26⁽³⁾</u>	<u>2.00⁽³⁾</u>	<u>1.94⁽⁴⁾</u>	<u>1.92⁽⁴⁾</u>	<u>1.90⁽⁴⁾</u>	<u>1.92⁽⁴⁾</u>
<i>speed-up</i>	+17 %	+16 %	+2.7 %	-1.3 %	-2.2 %	-2.0 %	-2.5 %	-2.5 %	-1.0 %
EPSM	6.67	25.55	2.77	2.16	1.91	1.91	1.90	1.83	1.86
TSO q	5.41 ⁽⁵⁾	3.90 ⁽⁵⁾	3.29 ⁽⁵⁾	2.59 ⁽⁵⁾	2.17 ⁽⁵⁾	-	-	-	-

Table 5. Experimental results on a natural language sequence.

m	4	8	16	32	64	128	256	512	1024
AOSO q	11.14 ⁽²⁾	4.58 ⁽⁴⁾	3.89 ⁽⁴⁾	3.76 ⁽⁴⁾	3.16 ⁽⁶⁾	3.16 ⁽⁶⁾	3.18 ⁽⁶⁾	3.21 ⁽⁶⁾	3.16 ⁽⁶⁾
BNDM q	12.30 ⁽⁴⁾	4.35 ⁽⁴⁾	3.17 ⁽⁴⁾	2.49 ⁽⁴⁾	2.53 ⁽⁴⁾	2.52 ⁽⁴⁾	2.51 ⁽⁴⁾	2.54 ⁽⁴⁾	2.51 ⁽⁴⁾
BSDM q	4.73 ⁽²⁾	3.85 ⁽²⁾	<u>2.86⁽⁴⁾</u>	<u>2.35⁽⁴⁾</u>	<u>2.20⁽⁴⁾</u>	<u>2.09⁽⁴⁾</u>	<u>2.07⁽⁴⁾</u>	<u>2.02⁽⁴⁾</u>	<u>2.00⁽⁴⁾</u>
BXS q	7.38 ⁽²⁾	4.85 ⁽²⁾	3.43 ⁽⁴⁾	2.59 ⁽⁴⁾	2.59 ⁽⁴⁾	2.64 ⁽⁴⁾	2.62 ⁽⁴⁾	2.62 ⁽⁴⁾	2.63 ⁽⁴⁾
EBOM	4.33	3.47	3.05	2.74	2.54	2.51	2.40	2.40	2.57
FSBNDM qs	4.66 ^(2,0)	3.55 ^(3,1)	2.77 ^(3,1)	2.39 ^(3,1)	2.39 ^(3,1)	2.38 ^(3,1)	2.41 ^(3,1)	2.42 ^(3,1)	2.43 ^(3,1)
KBNDM	7.84	5.49	4.22	3.59	3.28	3.08	3.04	3.03	3.03
SBNDM q	5.75 ⁽²⁾	4.18 ⁽²⁾	3.13 ⁽⁴⁾	2.43 ⁽⁴⁾	2.52 ⁽⁴⁾	2.50 ⁽⁴⁾	2.52 ⁽⁴⁾	2.51 ⁽⁴⁾	2.52 ⁽⁴⁾
FS- w	6.05 ⁽⁶⁾	4.25 ⁽⁶⁾	3.39 ⁽⁶⁾	2.89 ⁽⁶⁾	2.73 ⁽⁶⁾	2.54 ⁽⁶⁾	2.43 ⁽⁶⁾	2.40 ⁽⁶⁾	2.39 ⁽⁶⁾
FJS	7.06	25.33	3.68	2.95	2.96	2.81	3.18	3.42	3.83
HASH q	19.96 ⁽³⁾	8.34 ⁽³⁾	5.02 ⁽³⁾	3.68 ⁽⁵⁾	3.17 ⁽⁵⁾	2.95 ⁽⁵⁾	2.96 ⁽⁵⁾	2.76 ⁽⁵⁾	2.65 ⁽⁵⁾
ASKIP	10.17	7.78	5.25	3.65	2.97	2.89	3.14	3.77	5.90
IOM	9.37	6.67	5.26	4.38	3.96	3.73	3.47	3.30	3.20
WOM	9.98	7.01	5.28	4.32	3.91	3.53	3.25	3.11	3.02
SKIP q	4.62 ⁽²⁾	3.58 ⁽³⁾	<u>2.78⁽³⁾</u>	<u>2.30⁽³⁾</u>	<u>2.13⁽⁴⁾</u>	<u>2.07⁽⁴⁾</u>	<u>2.06⁽⁴⁾</u>	<u>1.97⁽⁴⁾</u>	<u>1.96⁽⁴⁾</u>
<i>speed-up</i>	+6.7 %	-3.2 %	-2.8 %	-2.1 %	-3.2 %	-1.0 %	-0.5 %	-2.5 %	-2.0 %
EPSM	6.72	26.36	2.86	2.13	1.94	1.94	1.92	1.86	1.87
TSO q	5.54 ⁽⁵⁾	4.05 ⁽⁵⁾	3.26 ⁽⁵⁾	2.61 ⁽⁵⁾	2.23 ⁽⁵⁾	-	-	-	-

algorithm, it obtains always the best results in the case of small alphabets. In such cases the gain in performance is up to 7 %. When the size of the alphabet increases the new algorithm maintain the best results only in the case of medium and long patterns (i.e. for $m \geq 32$).

The same behavior can be observed in the case of real data experimental results, where the new solution obtains the best running time in most cases. In the case of a genome sequence it is always the best choice, with a gain in performance up to 12 %. Such gain is less evident when the size of the

alphabet increases. It is up to 2.5 % when searching protein sequences and natural language texts.

5 Conclusions

In this paper we presented a simple, yet efficient, variant of the Skip-Search algorithm, based on condensed alphabets. Although such extension has been applied to many algorithms in recent years, it turns out that when applied to the Skip-Search algorithm, it produces a very fast searching procedure. It will be interesting to investigate whether the use of multiple hash function can reduce the number of false positives detected during the filtering phase.

References

1. Allauzen, C., Crochemore, M., Raffinot, M.: Factor oracle: A new structure for pattern matching. In: Bartosek, M., Tel, G., Pavelka, J. (eds.) SOFSEM 1999. LNCS, vol. 1725, pp. 295–310. Springer, Heidelberg (1999). http://dx.doi.org/10.1007/3-540-47849-3_18
2. Baeza-Yates, R., Gonnet, G.H.: A new approach to text searching. *Commun. ACM* **35**(10), 74–82 (1992). <http://doi.acm.org/10.1145/135239.135243>
3. Cantone, D., Faro, S.: Fast-search algorithms: New efficient variants of the boyer-moore pattern-matching algorithm. *J. Automata Lang. Comb.* **10**(5/6), 589–608 (2005)
4. Cantone, D., Faro, S.: Improved and self-tuned occurrence heuristics. In: Holub, J., Zdárek, J. (eds.) *Proceedings of the Prague Stringology Conference 2013*, Prague, Czech Republic, 2–4 September 2013, pp. 92–106. Department of Theoretical Computer Science, Faculty of Information Technology, Czech Technical University in Prague (2013). <http://www.stringology.org/event/2013/p09.html>
5. Cantone, D., Faro, S., Giaquinta, E.: A compact representation of nondeterministic (suffix) automata for the bit-parallel approach. *Inf. Comput.* **213**, 3–12 (2012). <http://dx.doi.org/10.1016/j.ic.2011.03.006>
6. Charras, C., Lecroq, T.: *Handbook of Exact String Matching Algorithms*. College Publications (2004)
7. Charras, C., Lecroq, T., Pehoushek, J.D.: A very fast string matching algorithm for small alphabets and long patterns (extended abstract). In: Farach-Colton [11], pp. 55–64. <http://dx.doi.org/10.1007/BFb0030780>
8. Crochemore, M., Czumaj, A., Gasieniec, L., Jarominek, S., Lecroq, T., Plandowski, W., Rytter, W.: Speeding up two string-matching algorithms. *Algorithmica* **12**(4/5), 247–267 (1994). <http://dx.doi.org/10.1007/BF01185427>
9. Durian, B., Chhabra, T., Ghuman, S.S., Hirvola, T., Peltola, H., Tarhio, J.: Improved two-way bit-parallel search. In: Holub, J., Zdárek, J. (eds.) *Proceedings of the Prague Stringology Conference 2014*, Prague, Czech Republic, 1–3 September 2014, pp. 71–83. Department of Theoretical Computer Science, Faculty of Information Technology, Czech Technical University in Prague (2014)
10. Ďurian, B., Peltola, H., Salmela, L., Tarhio, J.: Bit-parallel search algorithms for long patterns. In: Festa, P. (ed.) *SEA 2010*. LNCS, vol. 6049, pp. 129–140. Springer, Heidelberg (2010)

11. Farach-Colton, M. (ed.): CPM 1998. LNCS, vol. 1448. Springer, Heidelberg (1998)
12. Faro, S., Külekci, M.O.: Fast and flexible packed string matching. *J. Discrete Algorithms* **28**, 61–72 (2014). <http://dx.doi.org/10.1016/j.jda.2014.07.003>
13. Faro, S., Lecroq, T.: Efficient variants of the backward-oracle-matching algorithm. In: Holub, J., Ždárek, J. (eds.) *Proceedings of the Prague Stringology Conference 2008*, pp. 146–160. Czech Technical University in Prague, Czech Republic (2008)
14. Faro, S., Lecroq, T.: The exact string matching problem: a comprehensive experimental evaluation. *CoRR* abs/1012.2547 (2010)
15. Faro, S., Lecroq, T.: A fast suffix automata based algorithm for exact online string matching. In: Moreira, N., Reis, R. (eds.) *CIAA 2012*. LNCS, vol. 7381, pp. 149–158. Springer, Heidelberg (2012). http://dx.doi.org/10.1007/978-3-642-31606-7_13
16. Faro, S., Lecroq, T.: A multiple sliding windows approach to speed up string matching algorithms. In: Klasing, R. (ed.) *SEA 2012*. LNCS, vol. 7276, pp. 172–183. Springer, Heidelberg (2012)
17. Faro, S., Lecroq, T.: The exact online string matching problem: A review of the most recent results. *ACM Comput. Surv.* **45**(2), 13 (2013). <http://doi.acm.org/10.1145/2431211.2431212>
18. Fredriksson, K., Grabowski, S.: Practical and optimal string matching. In: Consens, M.P., Navarro, G. (eds.) *SPIRE 2005*. LNCS, vol. 3772, pp. 376–387. Springer, Heidelberg (2005). http://dx.doi.org/10.1007/11575832_42
19. Karp, R.M., Rabin, M.O.: Efficient randomized pattern-matching algorithms. *IBM J. Res. Dev.* **31**(2), 249–260 (1987)
20. Knuth, D.E., Morris, J.H., Pratt, V.R.: Fast pattern matching in strings. *SIAM J. Comput.* **6**(1), 323–350 (1977)
21. Lecroq, T.: Fast exact string matching algorithms. *Inf. Process. Lett.* **102**(6), 229–235 (2007). <http://dx.doi.org/10.1016/j.ipl.2007.01.002>
22. Navarro, G., Raffinot, M.: A bit-parallel approach to suffix automata: Fast extended string matching. In: Farach-Colton [11], pp. 14–33. <http://dx.doi.org/10.1007/BFb0030778>
23. Yao, A.C.: The complexity of pattern matching for a random string. *SIAM J. Comput.* **8**(3), 368–387 (1979). <http://dx.doi.org/10.1137/0208029>